However, before going down this path, make sure you're getting a real benefit—it's a lot of extra code to write and to maintain.

Standard Protocols and Coercions

Although not technically part of the syntax of the language, the Ruby interpreter and standard library use various protocols to handle issues of type conversion that other languages would deal with using the type system. That is to say that the Ruby interpreter looks for certain standard method names and, if those names exist, uses the methods to convert an arbitrary class to a standard type.

Some objects have more than one natural representation. For example, you may be writing a class to represent Roman numbers (I, II, III, IV, V, and so on). This class would not be implemented as a subclass of Integer, because its objects are representations of numbers, not numbers in their own right. At the same time, they do have an integer-like quality. It would be nice to be able to use objects of our Roman number class wherever Ruby was expecting to see an integer.

To do this, Ruby has the concept of *conversion protocols*—an object may elect to have itself converted to an object of another class. Ruby has two different ways of looking at this kind of conversion.

An *explicit* conversion is triggered in the code deliberately by calling the conversion method. The intent here is to say that the original type is not the same as the type being converted to, but that the method gives a reasonable representation of the data in the new type. By convention, explicit conversion methods have short names, like to_s for Strings, to_h for Hash, and to_i for Integer.

These conversion methods are not particularly strict. If an object has any kind of decent representation as a string, for example, it will probably have a to_s method. Our RomanNumeral class would probably implement to_s in order to return the string representation of a number ("VII", for instance).

An *implicit* conversion is triggered by the Ruby interpreter as part of using new objects in the same context where standard types are expected. This form of conversion function uses methods with longer names such as to_str and to_int (though there are some cases where shorter names are used for implicit conversions for historical reasons).

The implicit conversions are stricter. You implement them only if your object can naturally be used every place a string or an integer (or whatever the original type) could be used. For example, our Roman number objects have a clear representation as an integer and so should implement to_int. When it comes to stringiness, however, we have to think a bit harder.

Roman numbers clearly have a string representation, but are they strings? Should we be able to use them wherever we can use a string itself? No, probably not. Logically, they're a representation of a number. You can represent them as strings, but they aren't plug-and-play-compatible with strings. For this reason, a Roman number won't implement to_str—it isn't really a string. Just to drive this home, Roman numerals can be converted to strings using to_s, but they aren't inherently strings, so they don't implement to_str.

To see how this works in practice, let's look at opening a file. The first parameter to new can be either an existing file descriptor (represented by an integer) or a filename to open. However, Ruby doesn't simply look at the first parameter and check whether its type is Fixnum